

# Complexity-Driven Construction of Sparse Hierarchies for Implicit Constructive Solid Geometry

Egor Chulkov  
Oleg V. Vasilyev

Center for Materials Technologies, Skolkovo Institute of Science and Technology, Moscow, Russia  
Egor.Chulkov@skoltech.ru, O.Vasilyev@skoltech.ru

Preprint

## Abstract

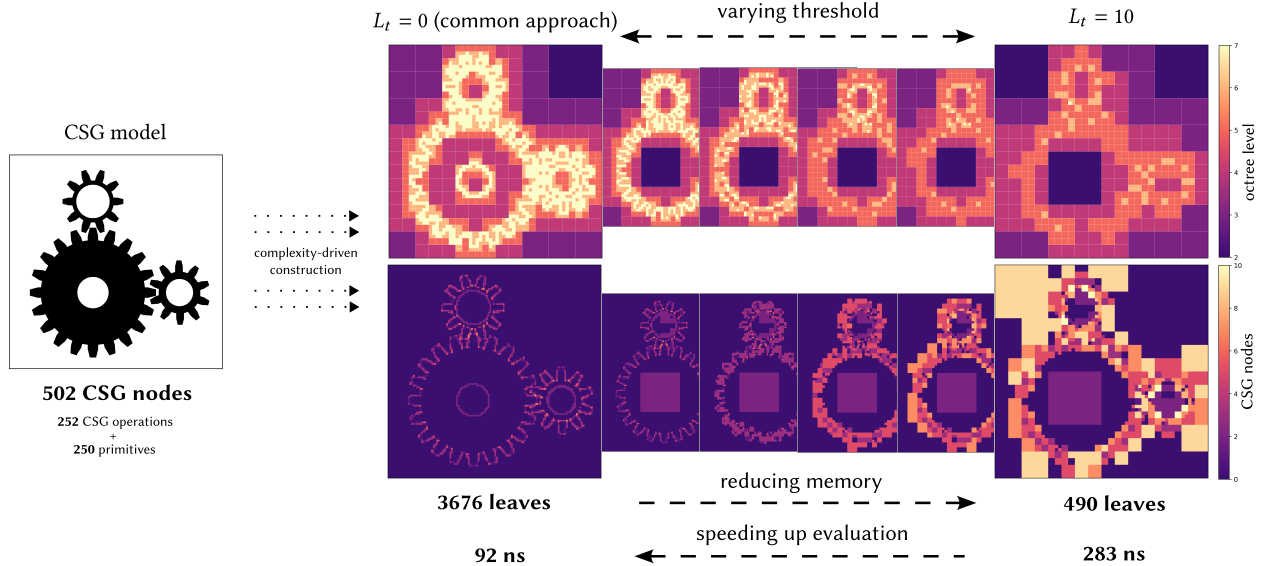
A complexity-driven acceleration method for implicit constructive solid geometry (CSG) models composed of closed-form primitives and set-theoretic operations is presented. Unlike existing sparse hierarchies that regulate memory and performance via spatial rules despite runtime being governed by expression complexity, in the proposed method spatial subdivision is tied directly to local complexity, given by the number of operations and primitives in the pruned expression of a voxel. The global CSG expression is pruned independently for each voxel using guaranteed bounds from range analysis, with refinement terminated when the local complexity falls below a user-defined threshold. The resulting hierarchies adapt to both geometric detail and the algebraic structure of CSG expressions, effectively decoupling evaluation cost from spatial resolution. Compared to conventional max-depth approaches, sparser hierarchies with equivalent local complexity are generated, reducing traversal overhead and memory while preserving the surface. Experiments across diverse detailed constructive scenes demonstrate more predictable scaling behavior and improved end-to-end performance.

**Keywords:** implicit CSG, complexity-driven construction, adaptive subdivision, sparse octrees, function representation

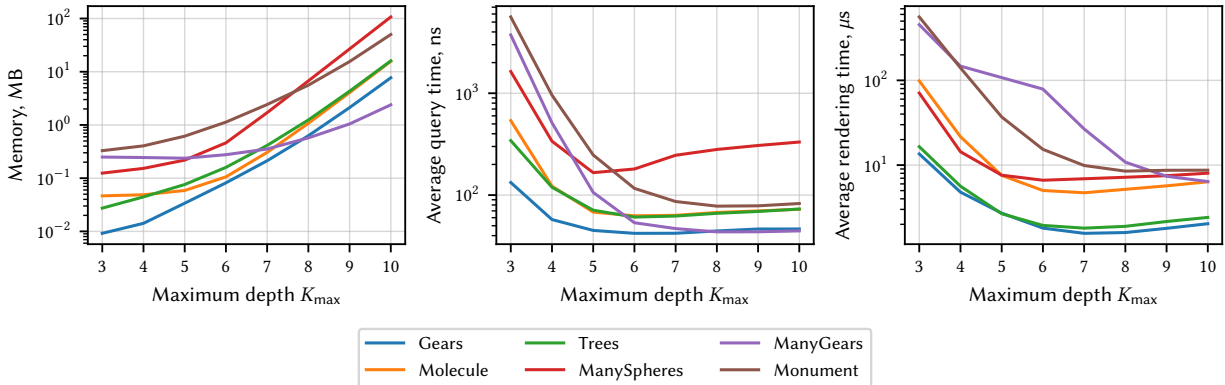
## 1 Introduction

Since the formal introduction of constructive solid geometry (CSG) by Ricci in 1973, which established a theoretical basis for solid modeling, implicit (or functional) representation has received relatively little attention as a unified geometric modeling paradigm. Early developments focused on establishing the mathematical formalism of R-functions Rvachev (1975) and exploring modeling operations for implicit geometric primitives Pasko et al. (1995); Enszt et al. (1998). Until recently, the general field of geometric modeling was dominated by explicit boundary representations Requicha (1980) due to their intuitive control and efficient visualization capabilities. Consequently, implicit representation was typically utilized in specialized settings, including robust solid modeling and classification via interval-based subdivision Duff (1992), voxel-based and sparse volumetric processing Frisken et al. (2000); Museth (2013), and visualization / interactive systems that evaluate closed-form implicit expressions efficiently Keeter (2020). More recently, interest has been renewed by digital fabrication and design workflows that benefit from exact Boolean operations, interior-aware representations, and scalable field evaluation Maltsev et al. (2021); Uchytel and Storti (2023); Popov et al. (2021).

These advantages stem directly from the mathematical nature of the representation itself. Implicit modeling represents a solid object via a scalar field  $F(\mathbf{x})$ , where the boundary of the object is given by an



**Figure 1.** The memory–performance trade-off in complexity-driven construction of implicit CSG models. Increasing the threshold  $L_t$  terminates boundary refinement earlier, yielding sparser hierarchies with reduced traversal overhead (fewer leaves), but retaining more complex pruned local expressions per leaf. Decreasing  $L_t$  forces finer subdivision to simplify local expressions, eventually approaching the conventional max-depth baseline at  $L_t = 0$ .



**Figure 2.** Effect of the maximum octree depth  $K_{\max}$  on memory and runtime. Across scenes (Fig. 7), the memory (left) grows steadily with  $K_{\max}$ , while the average query time (middle) and rendering time (right) may decrease, flatten, or increase depending on the scene.

isosurface, typically  $F(\mathbf{x}) = 0$ . In contrast to boundary representation, Boolean operations are naturally expressed as function compositions, and operations such as blending, offsets, and procedural detailing can be defined directly on the field Pasko et al. (2002, 2011). This family includes function representation (F-rep) Pasko et al. (1995), signed distance fields (SDFs), and hierarchical implicit systems such as BlobTrees Wyvill et al. (1999), with more recent systems adopting general procedural node graphs Riso et al. (2024). In F-rep, CSG operations are commonly encoded through R-functions, while SDF-based methods use min/max compositions for hard CSG and smooth variants for blending.

Rendering implicitly defined objects is typically performed by either polygonization or ray-based methods. Polygonization, e.g., Marching Cubes Lorensen and Cline (1987) and Dual Contouring Ju et al. (2002), is the fastest route to interactive visualization but introduces discretization error and requires regeneration

when the model changes. Ray-based methods instead evaluate the field directly along view rays, rendering the exact surface via sphere tracing for SDFs Hart (1996) or root finding with interval-based range tests for general F-rep Snyder (1992); Knoll et al. (2009).

A major practical bottleneck is the cost of repeated function evaluation: ray-casting and root finding may require thousands of evaluations per pixel. Similarly, in geometric modeling, point classification often dominates the runtime for complex implicit CSG models. To accelerate these computations, two complementary approaches have been pursued. The first uses spatial hierarchies that store near-surface information to avoid evaluating the full expression in empty space, for instance, through adaptive sampling Frisken et al. (2000); Uchytel and Storti (2023). The second is expression pruning, which removes operations that cannot affect the result in a local region, resulting in locally simplified expressions that are cheaper to evaluate Keeter (2020); Barbier et al. (2025).

Despite these advances, existing sparse hierarchies typically balance memory and performance by enforcing a fixed maximum resolution, such as a maximum octree depth. This parameter requires careful tuning: deeper trees reduce the average cost of evaluating the *pruned local expression*, but increase hierarchy traversal overhead and memory. In practice, the optimal choice is scene-dependent, and increasing maximum depth may either improve or degrade end-to-end performance depending on how the local evaluation cost varies across the scene. This makes the performance outcome less predictable and complicates parameter selection, as illustrated across scenes in Fig. 2.

This paper introduces a complexity-driven construction paradigm that directly couples the topology of the hierarchy to the local algebraic structure of the implicit CSG model. Unlike fixed-depth or geometric rules, which remain agnostic to the underlying evaluation cost of the CSG expression, the proposed approach employs a termination rule that forces spatial subdivision to adapt explicitly to local complexity. Hereafter, the *complexity* of a CSG expression is given by the number of operations and primitives it contains, serving as a proxy for its evaluation cost, while the *local complexity* is defined as the complexity of the pruned expression within a voxel. Subdivision of a voxel stops once its local complexity falls below a user-defined *threshold*  $L_t$ . Intuitively, the topology of the hierarchy is no longer an input parameter, but an emergent property derived from the distribution of complexity inherent to the implicit CSG model.

Figure 1 previews the resulting behavior. Increasing the threshold  $L_t$  terminates subdivision of boundary voxels earlier, substantially reducing octree occupancy (and thus memory and traversal cost), but retaining larger pruned local expressions that increase per-query evaluation cost. Decreasing  $L_t$  has the opposite effect, approaching the conventional max-depth behavior, which is recovered as the special case  $L_t = 0$ . Overall, the proposed threshold-based strategy provides direct control over the memory–performance trade-off, leading to more predictable acceleration and substantially reduced memory for a given runtime target in highly detailed scenes.

The contributions of this work are summarized as follows:

- A *threshold-based stopping rule* is proposed for adaptive spatial subdivision, terminating refinement based on the local complexity of the implicit CSG model. The proposed strategy provides direct control over the memory–performance trade-off via a single complexity parameter, with the standard max-depth strategy recovered as a special case.
- A criterion-agnostic pruning approach for implicit CSG models is introduced that constructs simplified local expressions optimized for near-surface tasks, such as ray-casting and contouring, as well as point membership classification, by skipping operations provably irrelevant to the result within each voxel.
- A systematic evaluation of the proposed threshold-based stopping rule is presented, benchmarking the method against the conventional max-depth termination rule and state-of-the-art pruning criteria in terms of local complexity, memory, query and rendering time, and construction time.

This work focuses on spatial acceleration for constructive implicit modeling in computer graphics and computer-aided design (CAD). The paper studies implicit solids built from closed-form primitives combined by hard CSG operators, targeting workloads dominated by repeated field queries. The scope is limited to algorithmic comparisons between pruning criteria and termination rules within a common backend framework. As a result, the reported performance figures represent relative comparisons among the evaluated methods, rather than absolute timings. A parallel or GPU implementation, a substantial undertaking, is deferred to future work.

A review of related work on accelerating evaluation of implicit functions through spatial hierarchies and expression pruning is provided in Section 2. A novel threshold-based stopping rule is proposed in Section 3, followed by a description of the pruning approach in Section 4. The rendering pipeline is outlined in Section 5. Section 6 presents the experimental results and comparisons, with discussion in Section 7. Future work is outlined in Section 8, and conclusions and limitations are covered in Section 9.

## 2 Related Work

**Spatial hierarchies and acceleration** Spatial acceleration is fundamental for efficient evaluation of implicit representations. Adaptively Sampled Distance Fields (ADFs) store sampled data in octrees refined near the surface Frisken et al. (2000), while Sparse Voxel Octrees (SVOs) enable efficient ray traversal via empty-space skipping Laine and Karras (2010). In production environments, VDB Museth (2013) and its GPU-optimized variants, such as NanoVDB Museth (2021) and GVDB Hoetzlein (2016), provide robust sparse topological management. However, in most existing systems, the construction of the acceleration structure is driven primarily by geometric rules, such as surface proximity or a fixed maximum resolution, rather than the computational cost of the underlying field evaluation.

**Interval analysis and expression pruning** Interval arithmetic (IA) provides the rigorous inclusion guarantees necessary for robust root finding and classification Snyder (1992). Duff (1992) pioneered the use of IA for spatial subdivision, introducing *sign-preserving* pruning of CSG trees, such that the resulting expression has the same sign as the original field, so the inside/outside classification is preserved exactly even if the field value changes. More recently, Keeter (2020) extended this to *value-preserving* pruning, simplifying min/max expressions for interactive rendering, where the resulting expression has both the same sign and the same value as the original field. Both methods rely on IA, which suffers from overestimation due to the dependency problem. Tighter bounds can be achieved with alternative arithmetics such as affine Stolfi and de Figueiredo (2003), revised affine Fryazinov et al. (2010), or Taylor models Makino and Berz (2003), though at a higher computational cost. Other recent methods avoid interval analysis altogether, relying instead on Lipschitz and distance-based bounds. Zanni (2024) pruned the BlobTree on-the-fly during interactive rendering using Lipschitz-based bounding volumes, while Barbier et al. (2025) utilized Lipschitz bounds to accelerate sphere tracing by pruning primitive-based SDFs that combine both hard and smooth CSG operators. Hubert-Brierre et al. (2025) reduced evaluation cost by embedding distance-driven proxy and level-of-detail nodes directly in the construction tree, without spatial subdivision. However, in all of these methods the resulting structure is governed by spatial resolution or surface distance. In contrast, the proposed approach allows the complexity of the pruned expression itself to determine the topology of the hierarchy.

**Termination rules in adaptive subdivision** Most hierarchical approaches require a stopping rule to balance competing demands on memory, precision, and performance. Common choices include a fixed maximum depth (standard in SVOs/VDB), geometric error thresholds (ADFs), or termination once a region is provably inside or outside (interval subdivision). However, these rules can be ill-suited for constructive models with



non-uniform complexity: deep refinement is wasteful in computationally simple regions, yet potentially insufficient in complex ones. While heuristics such as “one primitive per voxel” have been proposed Dyllong and Grimm (2008), they lack a formalized mechanism to balance memory against runtime. In contrast, the proposed method formalizes local complexity as a tunable constraint. This decouples subdivision from fixed geometric targets, providing guarantees on the local evaluation cost.

### 3 Threshold-based stopping rule

Standard construction algorithms build hierarchies such as octrees or  $N^3$ -trees by recursively subdividing space until either a voxel is proven empty or a maximum resolution is reached, without accounting for the cost of evaluating the implicit CSG model. To address this limitation, a termination rule governed by local complexity rather than spatial resolution is proposed. This section formalizes the rule and analyzes the resulting trade-off between traversal and evaluation cost.

**Complexity-driven termination** Let  $L(\mathcal{V})$  denote the local complexity of voxel  $\mathcal{V}$ , defined as the number of operations and primitives in its pruned expression. Subdivision is terminated when

$$L(\mathcal{V}) \leq L_t, \quad (1)$$

where  $L_t$  is a user-defined threshold. This rule ensures that subdivision continues only while it remains computationally advantageous: once the local expression is sufficiently simplified, further refinement is stopped, avoiding unnecessary increases in memory usage and traversal depth.

This behavior is illustrated qualitatively in Fig. 1. Larger values of  $L_t$  terminate refinement earlier, reducing the leaf count but retaining more complex expressions in boundary voxels. Conversely, decreasing  $L_t$  forces finer subdivision, approaching the behavior of a conventional max-depth hierarchy as  $L_t \rightarrow 0$ . This formulation generalizes existing heuristics; for instance, targeting “one primitive per voxel” Dyllong and Grimm (2008) is equivalent to setting  $L_t \approx 1$ . As demonstrated in Section 6, intermediate thresholds typically yield the best performance, and the threshold parameter  $L_t$  directly governs the memory–performance trade-off. Guidelines for selecting  $L_t$  are provided in Section 7.

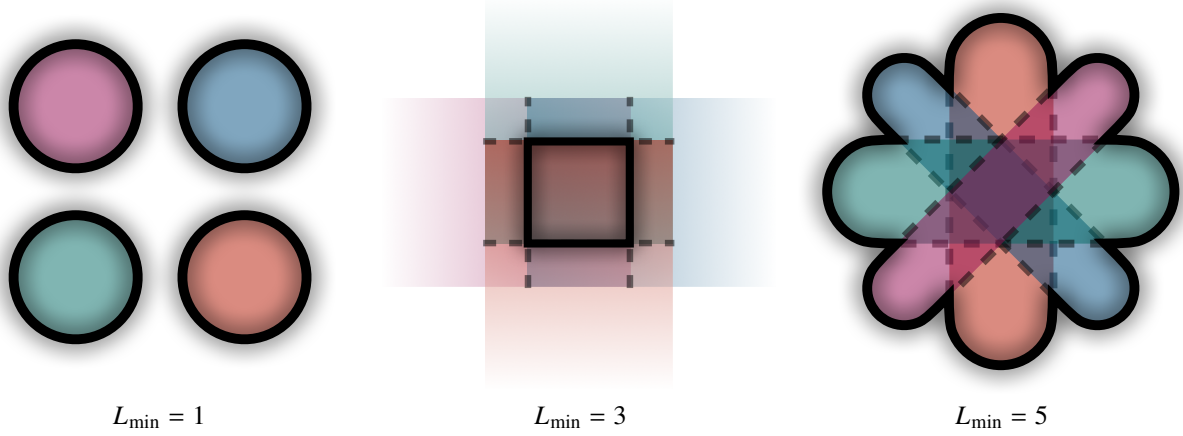
Since  $L(\mathcal{V})$  serves only as a proxy for the evaluation cost, a model-specific estimate can be chosen to better match the representation. For closed-form F-rep and SDFs, the number of arithmetic operations provides a natural measure, while for tree-structured representations similar to CSG trees, such as BlobTrees, the count of primitives and operators is a straightforward alternative.

In the benchmark scenes used in this work (Section 6), primitives are simple closed-form functions, such as spheres, half-spaces, cylinders, and disks, whose computational complexity is equivalent to a few arithmetic operations, and evaluating a single primitive is far cheaper than evaluating the full CSG expression. Since the evaluation cost of primitives and operators is comparable, the unweighted local complexity is found to be a practical proxy for evaluation cost.

For models whose primitive functions vary widely in computational cost, the same threshold rule can be applied using a weighted local complexity, with more expensive primitives assigned larger weights, without changing the pruning criterion or the threshold-based construction logic.

**Traversal–evaluation trade-off** Once the hierarchy has been constructed, the evaluation of an implicit function at a query point  $\mathbf{x}$  involves two distinct costs: (i) traversing the spatial structure to locate the leaf voxel  $\mathcal{V}_{\mathbf{x}}$  containing  $\mathbf{x}$ , and (ii) evaluating the pruned expression within that voxel. The query time  $T(\mathbf{x})$  is therefore expressed as

$$T(\mathbf{x}) = T_1(\mathbf{x}) + T_2(\mathbf{x}), \quad (2)$$



**Figure 3.** Examples of the maximum unprunable complexity in the limiting case  $K_{\max} \rightarrow \infty$ . Different colors denote distinct primitives.

where  $T_1$  is the traversal time and  $T_2$  is the evaluation time.

For an adaptive hierarchy, the traversal cost  $T_1$  grows with the depth of the leaf voxel  $K(\mathcal{V}_x)$ , while the evaluation cost  $T_2$  depends on the local complexity  $L(\mathcal{V}_x)$ . A simplified linear cost model is therefore

$$T(\mathbf{x}) \approx C_1 K(\mathcal{V}_x) + C_2 L(\mathcal{V}_x) + C_3, \quad (3)$$

where  $C_1$ ,  $C_2$ , and  $C_3$  are implementation-dependent constants representing traversal cost per level, evaluation cost per complexity unit, and fixed overheads, respectively.

A natural question is whether a similar memory–performance trade-off can be achieved simply by tuning the maximum depth. As shown in the introduction (Fig. 2), this strategy is unreliable, since finer refinement lowers evaluation cost ( $T_2$ ) while raising traversal cost ( $T_1$ ), with the net outcome varying by scene.

This tuning challenge becomes even less transparent in production-grade sparse volume structures, such as  $N^3$ -trees and VDB-style grids, where resolution and topology are governed by multiple interacting parameters Museth (2013); Uchytel and Storti (2023). In contrast, the proposed method introduces a single complexity parameter,  $L_t$ . Adjusting  $L_t$  generally forces changes in hierarchy depth to satisfy the complexity constraint. While this structural adaptation is often negligible for large thresholds, it becomes significant for small thresholds, where the hierarchy is forced to expand deeply, causing traversal costs to rise. This fundamental trade-off exists regardless of the specific spatial data structure. Thus, although an octree is evaluated in this work, the threshold-based logic is equally applicable to other hierarchical grid structures.

**Maximum unprunable complexity** Local complexity cannot be reduced arbitrarily, since even under the most aggressive threshold,  $L_t = 0$ , which refines every boundary voxel to the maximum depth  $K_{\max}$ , an irreducible complexity persists. At this finest resolution, the local complexity is bounded above by

$$L_{\min} = \max_{\mathcal{V} \in \mathcal{H}_{\max}} L(\mathcal{V}), \quad (4)$$

where  $\mathcal{H}_{\max}$  is the set of voxels at the finest available resolution. This *maximum unprunable complexity* reflects local structure that even the smallest voxels cannot resolve, approaching 1 for simple models and remaining high for dense configurations (Fig. 3).

**Maximum residual complexity** The worst-case per-leaf complexity, called the *maximum residual complexity*, is therefore

$$\tilde{L} = \max(L_t, L_{\min}), \quad (5)$$

since lowering  $L_t$  below  $L_{\min}$  cannot simplify the most complex voxels any further and merely drives them to the finest level  $K_{\max}$ . In practice,  $\tilde{L}$  does not need to be known in advance, as it is simply the largest local complexity encountered among the leaves of the hierarchy and can be obtained directly during construction.

Given that the threshold rule bounds the complexity of every voxel, the cost of evaluating any pruned expression is bounded as well, leading to a worst-case *arithmetic speedup*

$$S_{\text{est}} = \frac{N}{\tilde{L}}, \quad (6)$$

where  $N$  is the *global complexity*, defined as the complexity of the entire unpruned CSG expression. This guarantee is unavailable to a purely spatial rule, since limiting the maximum resolution alone leaves the residual complexity, and hence the evaluation cost, uncontrolled. When the threshold is achievable everywhere ( $L_t \geq L_{\min}$ ), the bound reduces to  $N/L_t$  and is known a priori. Otherwise the floor  $\tilde{L}$  is computed during construction, and  $N/\tilde{L}$  is obtained once the hierarchy is built.

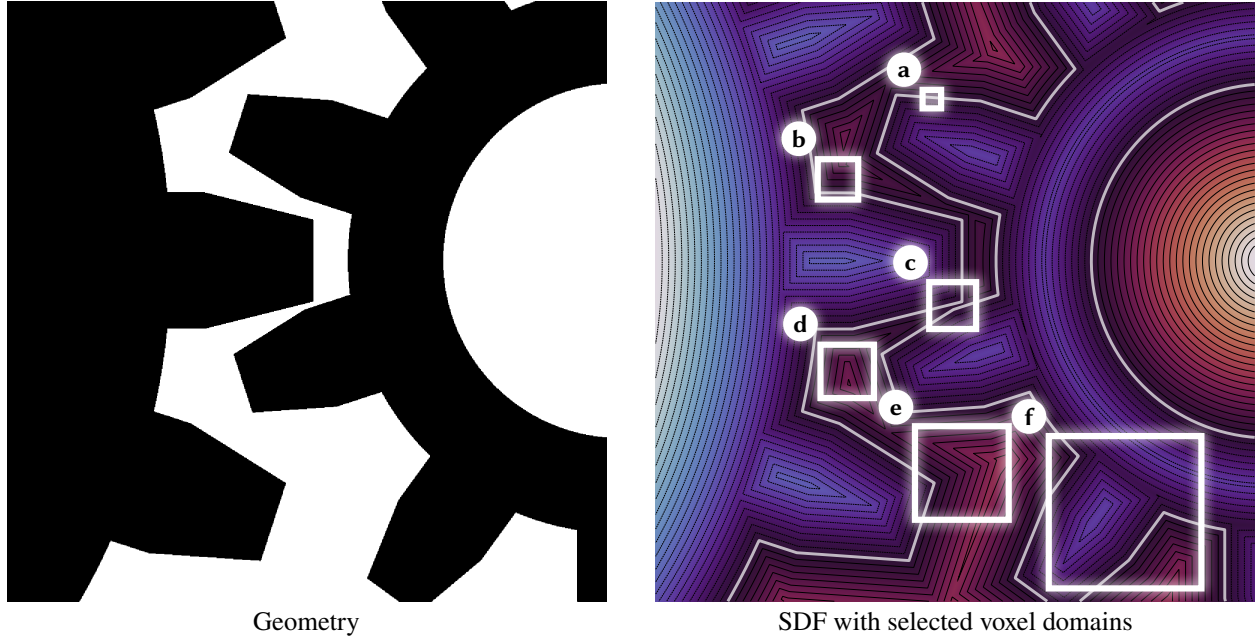
The bound (Eq. 6) is necessarily conservative, since it is based on the worst-case voxel and ignores traversal ( $T_1$ ), whereas most voxels are far simpler and most queries fall in empty space, where no expression is evaluated. Realized speedups therefore frequently exceed  $S_{\text{est}}$  by a substantial margin. For large thresholds, where the hierarchy is shallow and  $T_1$  is negligible,  $S_{\text{est}}$  also serves as a lower bound on the speedup in query time. For example, for *Trees* at  $L_t = 64$ , this lower bound is  $36\times$  (Table 2).

## 4 Pruning

This section presents the pruning and its implementation, integrated into the complexity-driven construction. Following prior spatial pruning methods Duff (1992); Keeter (2020); Barbier et al. (2025), the CSG expression is simplified independently within each voxel: a voxel that is provably uniform (entirely inside or outside) is certified as a constant label and incurs no further evaluation, and only boundary voxels retain a pruned local expression. All of these methods share a single mechanism, which estimates a conservative range of each subexpression over the voxel using range-based arithmetic and discards any subexpression whose removal provably preserves the result within that voxel. These methods differ only in how the range is estimated and in the rule that decides what to discard. Pruning is therefore treated as a criterion-agnostic approach with a single per-voxel algorithm whose range model and decision rule are interchangeable. The pruning criteria are described first (Section 4.1), followed by their implementation (Section 4.2).

**Table 1.** Pruning rules for binary operators under the Boolean, Keeter, and Lipschitz criteria. Each rule replaces a binary operator over a voxel  $\mathcal{V}$  with just one operand (union is max, intersection is min under the F-rep convention). The Boolean criterion tests each operand range  $[f_*, \overline{f}_*]$  against zero and preserves the sign. The Keeter and Lipschitz criteria keep the operand selected by the min/max and preserve the value. In the Lipschitz column,  $R$  denotes the radius of the voxel  $\mathcal{V}$  and  $f'_* \equiv f'_*(p)$ , where  $p$  is the voxel center.

|   | Operator   | Condition                                      |                                       |                       | Result |
|---|------------|------------------------------------------------|---------------------------------------|-----------------------|--------|
|   |            | Boolean                                        | Keeter                                | Lipschitz             |        |
| ① | $A \cup B$ | $\underline{f}_A > 0$ or $\overline{f}_B < 0$  | $\underline{f}_A \geq \overline{f}_B$ | $f'_A - f'_B \geq 2R$ | $A$    |
| ② |            | $\underline{f}_A < 0$ or $\underline{f}_B > 0$ | $\overline{f}_B \geq \underline{f}_A$ | $f'_B - f'_A \geq 2R$ | $B$    |
| ③ | $A \cap B$ | $\overline{f}_A < 0$ or $\underline{f}_B > 0$  | $\overline{f}_A \leq \underline{f}_B$ | $f'_B - f'_A \geq 2R$ | $A$    |
| ④ |            | $\underline{f}_A > 0$ or $\overline{f}_B < 0$  | $\underline{f}_B \leq \overline{f}_A$ | $f'_A - f'_B \geq 2R$ | $B$    |



| Method       | Domain   |          |          |          |           |           |
|--------------|----------|----------|----------|----------|-----------|-----------|
|              | <i>a</i> | <i>b</i> | <i>c</i> | <i>d</i> | <i>e</i>  | <i>f</i>  |
| Keeter       | <b>1</b> | 11       | 15       | <b>0</b> | 29        | 34        |
| Lipschitz    | <b>1</b> | 11       | 15       | 15       | 29        | 30        |
| Boolean (IA) | <b>1</b> | <b>3</b> | <b>7</b> | <b>0</b> | <b>11</b> | <b>14</b> |

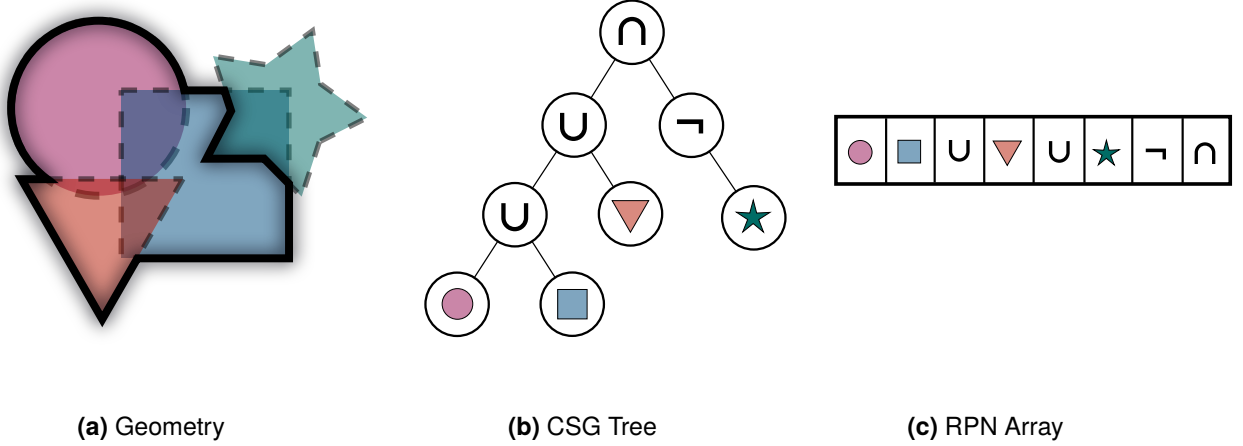
**Figure 4.** Pruning comparison on the 2D gears model. Left: Geometry of the CSG object. Right: Conservative SDF with six selected voxel domains. The table reports the local complexity (the number of operations and primitives remaining after pruning) for each domain, with bold values indicating the lowest local complexity.

#### 4.1 Criteria

A sign-preserving criterion retains the sign of the field over a voxel, and therefore the surface and the inside/outside classification, but not the field value. The instance used in this work is the *Boolean* criterion: each subexpression over a voxel  $\mathcal{V}$  is bounded by a conservative interval enclosure  $[\underline{f}_i, \overline{f}_i]$ , and, under the F-rep convention (positive inside), a union or intersection whose operand is certified entirely inside ( $\underline{f}_i > 0$ ) or outside ( $\overline{f}_i < 0$ ) reduces to that operand by the set-algebra rules of Table 1. The method of Duff (1992) is the special case that classifies primitives with interval arithmetic.

This classification applies to arbitrary subexpressions, so the range model can be freely chosen, e.g., interval Moore (1966), affine Stolfi and de Figueiredo (2003), revised affine Fryazinov et al. (2010), or Taylor models Makino and Berz (2003). A tighter model can certify a subexpression that interval arithmetic leaves undecided, such as a union of two boundary operands that is nonetheless entirely inside, by capturing operand correlations that interval arithmetic discards. A branch is dropped as soon as its inside/outside state is determined, even if the field value would change. The rules depend only on this classification and are therefore independent of the R-function encoding Rvachev (1975) of the operators.

Value-preserving criteria, in contrast, retain the exact field value, so tasks that depend on field magnitudes, such as distance queries or SDF sphere tracing, behave exactly as on the original model. Keeter pruning Keeter (2020) and Lipschitz pruning Barbier et al. (2025) achieve this at min/max nodes by comparing the two operand ranges and discarding the operand that is never selected. The two methods differ only in the bound



**Figure 5.** Representation of the implicit CSG model: (a) the resulting 2D geometry constructed from primitives, (b) the corresponding CSG tree, and (c) the Reverse Polish Notation (RPN) array used as the contiguous memory layout for the pruning algorithm.

used, with interval arithmetic for Keeter and a 1-Lipschitz bound for Lipschitz. All three criteria are unified under a single pruning algorithm, parameterized by the range model and the decision rule (Table 1). Figure 4 compares these criteria on six voxels from a 2D gears model, where the Boolean criterion yields the lowest local complexity.

Value-preserving criteria can also prune smooth operators. Away from the blending region, a smooth operator reduces to the corresponding hard operator, and the branch-selection rule certifies this reduction from the operand ranges. The Boolean criterion, in contrast, is limited to hard operators, since blending operations require the exact field value, which the Boolean criterion does not preserve. Extending the Boolean criterion to smooth operators is discussed in Section 8.

For contouring tasks, the pruned expression must yield the same surface intersection points and surface normals as the original expression. The intersection points lie on the zero level set, which sign preservation retains exactly, and the unit normal at a regular surface point depends only on this level set, so both are preserved for any operator. The normal is obtained from the surface gradient. For a value-preserving criterion the gradient is preserved directly, since the entire field is preserved. For the Boolean criterion the field value away from the surface may change, so gradient preservation is not obvious. Nevertheless, for the exact min/max and standard R-function operators considered here, the discarded branch does not affect the gradient at the surface, so the full surface gradient is preserved at regular surface points (Appendix A). The pruned CSG expression therefore yields the same Hermite data, i.e., surface intersection points and normals, used by methods such as Dual Contouring Ju et al. (2002).

## 4.2 Implementation

**Local expression representation** The CSG expression is stored globally in Reverse Polish Notation (RPN) as a contiguous token array  $E = [e_1, \dots, e_N]$  of global complexity  $N$ , where each *token*  $e_i$  is a single primitive or operator (union, intersection, or negation). For a given voxel  $\mathcal{V}$ , the pruned local expression is represented by an *active index list*  $A = [a_1, \dots, a_L] \subseteq \{1, \dots, N\}$ , whose entries are global indices into  $E$ . Thus,  $A[p]$  is the global index of the active (non-pruned) token at local position  $p$ , and the local expression is the subsequence  $[e_{a_1}, \dots, e_{a_L}]$ . An illustration of the implicit CSG model together with the CSG tree and the corresponding RPN representation is given in Fig. 5.

Each primitive token denotes a scalar implicit function in closed form. Under the F-rep convention,

---

**Algorithm 1:** Pruning of a CSG expression within a voxel.

---

**Input** : token array  $E$ , voxel  $\mathcal{V}$ , active index list  $A_0$

**Output** IN, OUT, or BOUNDARY( $A$ ,  $|A|$ )

```

1  Function PruneVoxel( $E, \mathcal{V}, A_0$ ):
2     $A \leftarrow A_0$ 
3     $\mathbf{m} \leftarrow \mathbf{1}^{|A|}$                                  $\triangleright$  bitset over local positions in  $A$ 
4     $D \leftarrow \text{ARRAY}(|A|)$                          $\triangleright$  descriptor per local position in  $A$ 
5     $S \leftarrow \emptyset$                                  $\triangleright$  stack of local positions in  $A$ 
6    for  $p \leftarrow 1$  to  $|A|$  do
7       $i \leftarrow A[p]$ 
8       $\tau \leftarrow E[i]$ 
9      if  $\tau$  is a primitive then
10        $D[p] \leftarrow \text{PRIMDESCRIPTOR}(\tau, \mathcal{V})$ 
11        $\text{PUSH}(S, p)$ 
12      else if  $\tau$  is NEGATION then
13        $q \leftarrow \text{POP}(S)$ 
14        $D[p] \leftarrow \text{OPDESCRIPTOR}(\tau, D[q])$ 
15        $\text{PUSH}(S, p)$ 
16      else                                 $\triangleright \tau$  is UNION or INTERSECTION
17        $p_b \leftarrow \text{POP}(S)$                                  $\triangleright$  right operand
18        $p_a \leftarrow \text{POP}(S)$                                  $\triangleright$  left operand
19        $(p_{\text{keep}}, p_{\text{drop}}) \leftarrow \text{PRUNERULE}(\tau, D[p_a], D[p_b], \mathcal{V}, p_a, p_b)$      $\triangleright$  Table 1
20       if  $p_{\text{keep}} \neq \text{NONE}$  then
21          $\mathbf{m}[p] \leftarrow 0$                                  $\triangleright$  remove the operator token
22          $\text{PruneSubtree}(E, A, \mathbf{m}, p_{\text{drop}})$ 
23          $\text{PUSH}(S, p_{\text{keep}})$                                  $\triangleright$  keep the surviving operand
24       else
25          $D[p] \leftarrow \text{OPDESCRIPTOR}(\tau, D[p_a], D[p_b])$ 
26          $\text{PUSH}(S, p)$ 
27    $p_r \leftarrow \text{POP}(S)$ 
28    $s_r \leftarrow \text{GETSTATE}(D[p_r], \mathcal{V})$ 
29   if  $s_r \neq \text{BOUNDARY}$  then
30     return  $s_r$                                  $\triangleright$  uniform voxel
31   else
32      $A \leftarrow \text{COMPACT}(A, \mathbf{m})$ 
33     return BOUNDARY( $A$ ,  $|A|$ )

```

---

---

**Algorithm 2:** Deleting a subtree from the bitset  $\mathbf{m}$ .

---

**Input** : token array  $E$ , active index list  $A$ , bitset  $\mathbf{m}$ , local root position  $p_0$

1 **Function** PruneSubtree( $E, A, \mathbf{m}, p_0$ ):

2      $p \leftarrow p_0$

3      $r \leftarrow 1$  ▷ pending subexpressions to remove

4     **while**  $r > 0$  **do**

5          $\tau \leftarrow E[A[p]]$

6          $\mathbf{m}[p] \leftarrow 0$

7          $r \leftarrow r - 1$

8         **if**  $\tau$  is **NEGATION** **then**

9              $r \leftarrow r + 1$  ▷ add its operand

10         **else if**  $\tau$  is **UNION** **or** **INTERSECTION** **then**

11              $r \leftarrow r + 2$  ▷ add its two operands

12          $p \leftarrow \text{PrevBit}(\mathbf{m}, p)$

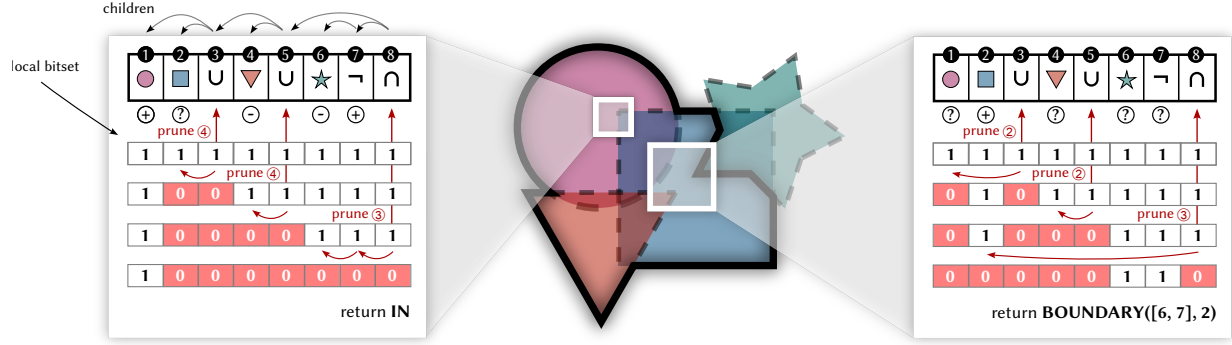
---

union is implemented as max and intersection as min. For the Boolean criterion, interval Moore (1966), affine Stolfi and de Figueiredo (2003), and revised affine Fryazinov et al. (2010) arithmetic are implemented to bound each subexpression over a voxel, with interval arithmetic adopted as the default, since it is the fastest and, in most cases, yields the same bounds as the affine variants. A new primitive can be added simply by supplying its closed-form expression.

**Single-pass pruning algorithm** The pruning is realized by a single pass over the active tokens (Algorithm 1) that mirrors ordinary stack-based RPN evaluation. Tokens are read in post-order (a depth-first, bottom-up traversal) while maintaining a stack of positions, except that the pass propagates a per-node *descriptor* and prunes operators instead of producing one scalar value. The descriptor summarizes a subexpression over the voxel, with an interval enclosure for the Boolean and Keeter criteria and the value at the voxel center for Lipschitz. A primitive computes its descriptor (PRIMDESCRIPTOR) and pushes its position. A negation, union, or intersection combines the descriptors of its operands (OPDESCRIPTOR). At a union or intersection, the operand descriptors are passed to the pruning rule of the corresponding criterion (Table 1, PRUNERULE). When the rule certifies that one operand determines the result, the operator is replaced by that surviving operand. The operator token is deactivated, the surviving operand is kept on the stack, and the subtree of the other operand is deleted (Algorithm 2). Otherwise, the operator is retained and its descriptor is pushed to the stack.

Subtree deletion is the step that executes a pruning decision, and it is the only one that traverses the RPN array backward. When a rule retains one operand of an operator, the dropped operand is the root of an entire subexpression that must be removed along with it. Rather than rewriting the token array  $E$ , a bitset  $\mathbf{m}$  is maintained over the positions of the active index list  $A$ , where each bit marks the corresponding token as active or discarded. A deletion simply clears bits and leaves  $E$  untouched. The reverse Polish order of the expression ensures that the discarded subexpression occupies a contiguous span ending at the dropped operand. The deletion therefore starts at that operand and steps backward using PREVBIT, which locates the previous active bit by scanning machine words and applying de Bruijn indexing within a word Leiserson and Prokop (1998).

The walk maintains a count of pending subexpressions still to remove, initialized to one. At each visited token, the bit is cleared and the count is decremented by one, accounting for that subexpression. If the token is an operator, its operands are still pending, so the count is incremented by the arity of the operator (one



**Figure 6.** Per-voxel pruning pass (Algorithm 1) on the model from Figure 5, illustrated for the Boolean criterion, for which only the sign of each RPN token matters. A token is labeled  $\oplus$  if  $f_i > 0$ ,  $\ominus$  if  $f_i < 0$ , and  $\odot$  otherwise. When a rule from Table 1 applies, the operator is replaced by one operand, the subtree of the dropped operand is cleared from the local bitset, and the step is marked with the rule used. The two voxels illustrate two possible outcomes: (left) a uniform voxel returns a constant label, and (right) a boundary voxel returns its surviving token indices and local complexity.

for a negation and two for a union or intersection), while a primitive leaves the count unchanged. The walk stops when the count returns to zero. That point marks the left end of the span, so the subtree is recovered without storing or following any tree pointers.

---

**Algorithm 3:** Complexity-driven octree construction.

---

**Input** : Root voxel  $V_0$ , token array  $E$ , threshold  $L_t$ , max depth  $K_{\max}$

**Output** Octree root  $R$ , maximum residual complexity  $\tilde{L}$

---

```

:
1  $A_0 \leftarrow [1, \dots, N]$                                 ▶ initial active index list
2  $\tilde{L} \leftarrow 0$                                           ▶ global
3  $R \leftarrow \text{BuildNode}(V_0, 0, A_0)$ 
4 return  $(R, \tilde{L})$ 
5 Function  $\text{BuildNode}(V, k, A_{\text{in}})$ :
6    $P \leftarrow \text{PruneVoxel}(E, V, A_{\text{in}})$ 
7   if  $P \in \{\text{IN}, \text{OUT}\}$  then
8     return  $\text{LEAF}(P)$                                      ▶ uniform voxel
9    $(A, L) \leftarrow P$                                      ▶ extract active index list and local complexity
10  if  $k = K_{\max}$  or  $L \leq L_t$  then
11     $\tilde{L} \leftarrow \max(\tilde{L}, L)$ 
12    return  $\text{LEAF}(A)$                                      ▶ boundary voxel
13   $u \leftarrow \text{INTERNALNODE}()$ 
14  foreach  $V_c$  in  $\text{SUBDIVIDE}(V)$  do
15     $c \leftarrow \text{BuildNode}(V_c, k + 1, A)$ 
16     $\text{ADDCHILD}(u, c)$ 
17  return  $u$ 

```

---

After the forward pass completes, the stack contains the local root position of the remaining expression. `GETSTATE` returns its state, which can be **IN**, **OUT**, or **BOUNDARY**. For the interval-based criteria (Boolean and Keeter), empty-space skipping is applied, certifying the voxel **IN** or **OUT** when the root interval excludes



zero, whereas for the Lipschitz criterion, far-field culling Barbier et al. (2025) is applied instead. If the voxel is uniform, its constant label is returned. Otherwise, the bits still set in  $\mathbf{m}$  are exactly the tokens that survive pruning, and they are gathered into the reduced active index list by `COMPACT`. Figure 6 traces the entire pass on the model from Figure 5.

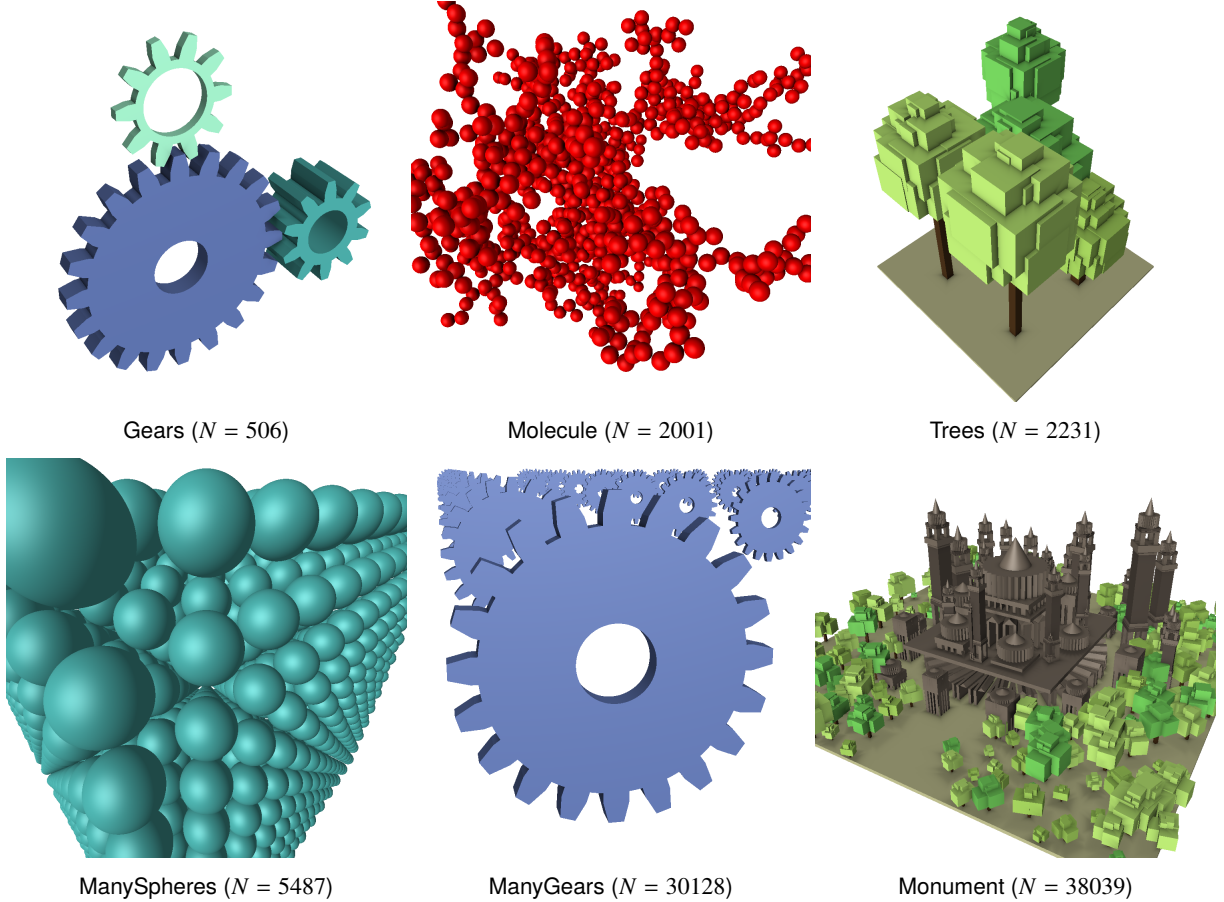
**Octree construction** Algorithm 3 details the integration of the pruning procedure into the octree construction process under the threshold-based stopping rule. For each voxel  $\mathcal{V}$ , pruning either certifies the state (**IN** / **OUT**) or returns the local complexity  $L(\mathcal{V})$ . Boundary voxels (**BOUNDARY**) terminate when either the target complexity is reached ( $L \leq L_t$ ) or the maximum depth  $K_{\max}$  is encountered. As discussed in Section 3, the maximum residual complexity  $\tilde{L}$  is determined during the construction process. As an optimization, each child voxel initializes pruning with the parent active index list  $A$ , so tokens already removed at coarser levels are not reconsidered.

## 5 Rendering

The implicit surface is visualized using standard first-hit ray-casting. Rays traverse the hierarchy in front-to-back order using a stack-based octree traversal algorithm. At each internal node, the visitation order of child voxels is determined by a 3D digital differential analyzer (DDA) Amanatides and Woo (1987). Child nodes are pushed to the stack in reverse intersection order to ensure correct front-to-back processing. Uniform voxels (**IN** and **OUT**) are skipped since they contain no zero level set, restricting candidate intersections to boundary voxels. Within a boundary voxel,  $F$  is evaluated by iterating over the active index list and fetching the corresponding tokens from the global token array. The first surface intersection is located using interval bisection Fryazinov et al. (2010); Snyder (1992). Unlike standard scalar bisection, which relies on bracketing a sign change, this method recursively subdivides the ray interval and rejects sub-segments where the function range strictly excludes zero ( $0 \notin [f, \bar{f}]$ ). This ensures robust detection of surfaces even when the voxel entry and exit points share the same sign. Surface normals are computed from  $\nabla F$  using the pruned local expression. As demonstrated in Appendix A, pruning preserves the surface gradient at regular surface points, and therefore, analytic normals computed from the pruned expression coincide with those of the original expression. Central differences are used for simplicity. Although finite differences may introduce small numerical errors in normals, no visible artifacts were observed in the benchmark scenes. The rendering times reported in Table 2 include hierarchy traversal, first-hit ray-casting, and normal computation.

## 6 Results

The proposed threshold-based stopping rule and pruning pipeline are benchmarked on six CSG scenes, shown in Fig. 7: *Gears*, *Molecule*, *Trees*, *ManySpheres*, *ManyGears*, and *Monument*. The scenes *Trees*, *Molecule*, and *Monument* follow the sharp-Boolean setup used in prior work Barbier et al. (2025), and the *Molecule* scene originates from Galin et al. (2020). The *Gears* scene is a CSG variant inspired by the gears example from Keeter (2020). *ManySpheres* and *ManyGears* serve as stress tests with many repeated primitives, designed to expose scaling trends. All timings are reported for a single-threaded CPU implementation, running on an Intel Xeon Silver 4214 at 2.20 GHz with 64 GB of RAM, and averaged over all evaluated points or pixels. The average query time  $\bar{T}$  is measured over  $10^6$  points sampled uniformly at random within the axis-aligned bounding box of each scene, while the rendering time  $\bar{T}_r$  is measured from a single fixed viewpoint per scene. Each reported timing is the mean of five runs. All benchmark scenes use hard CSG operators. The source code of the CPU reference implementation will be made publicly available upon publication.



**Figure 7.** Benchmark scenes with global complexities  $N$  used throughout the evaluation. Molecule is taken from Galin et al. (2020), and Trees and Monument from Barbier et al. (2025). Ambient occlusion and directional lighting are used for visual clarity.

Section 6.1 analyzes the impact of varying the threshold  $L_t$  on pruning aggressiveness, hierarchy size, and runtime. Section 6.2 compares the three pruning criteria (Boolean, Keeter Keeter (2020), and Lipschitz Barbier et al. (2025)) within the same framework and under the same threshold-based termination rule.

## 6.1 Threshold-based approach

The threshold-based construction is analyzed using the aggregate results in Table 2, obtained with the Boolean criterion and interval arithmetic. For each benchmark scene, the table reports the chosen threshold  $L_t$ , the resulting maximum residual complexity  $\tilde{L}$ , index storage  $M$ , the number of leaves, octree construction time  $T_c$ , average query time  $\bar{T}$ , and average rendering time per pixel  $\bar{T}_r$ , all at fixed maximum depth  $K_{\max} = 10$ . The reported  $\tilde{L}$  can fall a few tokens below  $L_t$  because subdivision is discrete and a voxel that exceeds  $L_t$  is split into eight smaller voxels whose local complexity may drop below  $L_t$  as fewer primitives overlap them. The row  $L_t = 0$  corresponds to the conventional max-depth strategy.

The maximum residual complexity  $\tilde{L}$  behaves as a worst-case floor. For high thresholds, lowering  $L_t$  directly lowers  $\tilde{L}$ . As  $L_t \rightarrow 0$ ,  $\tilde{L}$  saturates, since the worst-case voxel can no longer be simplified. This saturated value reflects the intrinsic local complexity of the model and need not track its global complexity  $N$ . For instance, *ManySpheres* contains thousands of primitives ( $N = 5487$ ) yet saturates at  $\tilde{L} = 1$ , because

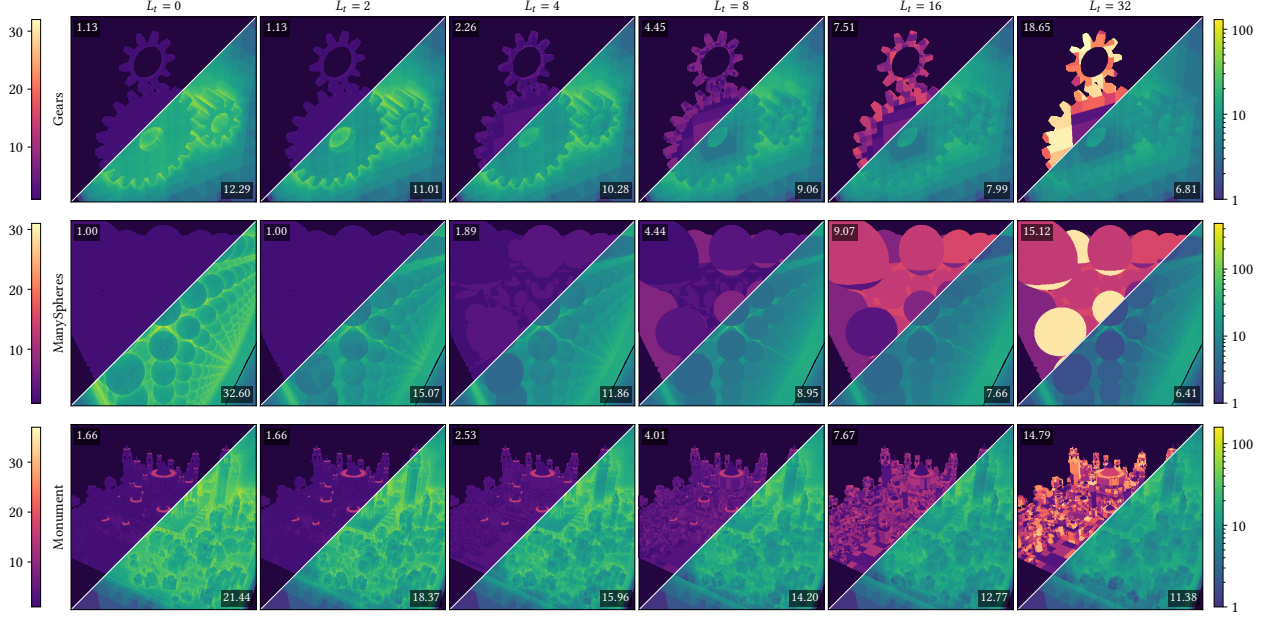
**Table 2.** Effect of the complexity threshold  $L_t$  on hierarchy size and runtime at fixed maximum depth  $K_{\max} = 10$ . For each scene, the table reports the maximum residual complexity  $\tilde{L}$ , index storage  $M$ , number of leaves, construction time  $T_c$ , average query time  $\bar{T}$ , and average rendering time per pixel  $\bar{T}_r$ . The case  $L_t = \infty$  denotes the original unpruned evaluation, for which hierarchy-dependent quantities are not reported. Bold values indicate the best runtime within each scene.

| Scene              | $L_t$    | $\tilde{L}$ | $M$ , MB | # Leaves   | $T_c$ , ms | $\bar{T}$ , ns | $\bar{T}_r$ , $\mu$ s |
|--------------------|----------|-------------|----------|------------|------------|----------------|-----------------------|
| <i>Gears</i>       | 64       | 64          | 0.01     | 148        | 1.93       | 253.14         | 16.90                 |
|                    | 32       | 32          | 0.01     | 442        | 2.86       | 120.10         | 8.81                  |
|                    | 16       | 16          | 0.03     | 1,471      | 5.05       | 61.41          | 3.53                  |
|                    | 8        | 8           | 0.05     | 6,000      | 10.16      | 45.29          | 2.30                  |
|                    | 4        | 5           | 0.32     | 85,338     | 67.49      | <b>42.49</b>   | 1.80                  |
|                    | 2        | 5           | 1.79     | 598,137    | 348.62     | 44.29          | <b>1.77</b>           |
|                    | 0        | 5           | 7.70     | 3,874,172  | 1848.58    | 46.49          | 2.03                  |
|                    | $\infty$ | —           | —        | —          | —          | 11599.10       | 220.74                |
| <i>Molecule</i>    | 64       | 63          | 0.05     | 841        | 18.43      | 430.88         | 32.33                 |
|                    | 32       | 31          | 0.06     | 1,849      | 22.27      | 218.77         | 19.70                 |
|                    | 16       | 15          | 0.06     | 3,991      | 26.07      | 119.88         | 10.68                 |
|                    | 8        | 7           | 0.08     | 9,528      | 32.14      | 86.79          | 7.23                  |
|                    | 4        | 5           | 0.16     | 32,145     | 51.19      | 74.13          | 5.26                  |
|                    | 2        | 5           | 1.74     | 539,120    | 334.58     | <b>70.16</b>   | <b>4.64</b>           |
|                    | 0        | 5           | 15.68    | 9,122,485  | 4209.32    | 72.55          | 6.35                  |
|                    | $\infty$ | —           | —        | —          | —          | 39370.60       | 2420.74               |
| <i>Trees</i>       | 64       | 61          | 0.04     | 631        | 8.26       | 322.54         | 12.00                 |
|                    | 32       | 31          | 0.05     | 1,793      | 13.32      | 197.18         | 6.43                  |
|                    | 16       | 15          | 0.07     | 5,559      | 20.47      | 124.53         | 4.01                  |
|                    | 8        | 11          | 0.15     | 22,723     | 40.93      | 95.39          | 2.73                  |
|                    | 4        | 11          | 0.63     | 155,492    | 130.49     | <b>63.83</b>   | 2.39                  |
|                    | 2        | 11          | 2.85     | 943,916    | 551.81     | 65.21          | <b>1.78</b>           |
|                    | 0        | 11          | 16.04    | 9,017,338  | 3741.05    | 72.98          | 2.41                  |
|                    | $\infty$ | —           | —        | —          | —          | 48837.20       | 944.47                |
| <i>ManySpheres</i> | 64       | 61          | 0.16     | 2,696      | 44.32      | 618.95         | 44.85                 |
|                    | 32       | 31          | 0.15     | 3,669      | 47.55      | 403.89         | 29.53                 |
|                    | 16       | 15          | 0.15     | 3,942      | 46.99      | 359.76         | 19.09                 |
|                    | 8        | 7           | 0.18     | 19,468     | 69.88      | 199.68         | 11.32                 |
|                    | 4        | 3           | 0.20     | 34,070     | 79.64      | <b>173.48</b>  | 7.97                  |
|                    | 2        | 1           | 0.30     | 126,491    | 136.48     | 179.63         | <b>6.37</b>           |
|                    | 0        | 1           | 107.04   | 65,688,288 | 30175.66   | 331.79         | 8.02                  |
|                    | $\infty$ | —           | —        | —          | —          | 109000.00      | 9709.60               |
| <i>ManyGears</i>   | 64       | 63          | 0.36     | 9,136      | 209.00     | 57.91          | 27.27                 |
|                    | 32       | 31          | 0.61     | 25,068     | 258.71     | 49.13          | 17.04                 |
|                    | 16       | 15          | 0.93     | 69,672     | 339.98     | 46.63          | 7.95                  |
|                    | 8        | 11          | 1.54     | 196,736    | 480.72     | 44.65          | 6.61                  |
|                    | 4        | 11          | 2.04     | 289,864    | 556.13     | <b>44.34</b>   | 6.38                  |
|                    | 2        | 11          | 2.18     | 323,828    | 577.73     | 44.85          | <b>6.36</b>           |
|                    | 0        | 11          | 2.39     | 453,160    | 626.95     | 44.50          | 6.40                  |
|                    | $\infty$ | —           | —        | —          | —          | 871698.00      | 35693.20              |
| <i>Monument</i>    | 64       | 75          | 1.19     | 16,304     | 293.20     | 276.12         | 26.26                 |
|                    | 32       | 75          | 2.57     | 66,375     | 447.51     | 152.58         | 16.28                 |
|                    | 16       | 75          | 6.02     | 289,794    | 845.19     | 112.03         | 11.22                 |
|                    | 8        | 75          | 11.22    | 879,397    | 1496.32    | 88.27          | 8.90                  |
|                    | 4        | 75          | 18.03    | 2,433,131  | 2617.62    | 82.04          | 8.07                  |
|                    | 2        | 75          | 31.37    | 6,109,678  | 4677.62    | <b>79.50</b>   | <b>7.98</b>           |
|                    | 0        | 75          | 50.06    | 15,905,870 | 8811.33    | 82.64          | 8.71                  |
|                    | $\infty$ | —           | —        | —          | —          | 1337820.00     | 50513.60              |

its local expression reduces to a single sphere. Conversely, a saturated floor that is unexpectedly high signals insufficient maximum resolution, so a larger  $K_{\max}$  can lower both  $\tilde{L}$  and the runtime. *ManyGears* illustrates this point. It repeats the *Gears* motif and would be expected to reach  $\tilde{L} = 5$ , but its larger domain leaves the voxels at depth  $K_{\max}$  too coarse to resolve each gear to the same complexity, raising the floor to  $\tilde{L} = 11$ .

The same quantity also yields a performance bound, as  $\tilde{L}$  gives the worst-case arithmetic speedup  $S_{\text{est}}$  (Eq. (6)). This bound is conservative, however, since it assumes the worst-case voxel for every query, yet it may be useful when a guarantee is required.

Beyond this worst-case bound, the threshold governs two measured quantities shown in Table 2, namely the memory and the query time. *Index storage* is reported as the total size of the active index lists, with each index occupying four bytes. Since the per-node cost is specific to the implementation, the octree node topology is not included in the index storage, and its scale is given by the leaf count. Lowering  $L_t$  forces deeper subdivision and stores more indices, so the index storage grows as  $L_t$  decreases and is largest at  $L_t = 0$ . Construction time closely tracks the memory (Table 2). As long as the threshold is achievable everywhere ( $\tilde{L} = L_t$ ), the index storage grows slowly because each subdivision is productive, replacing a leaf by children that prune to shorter index lists, so the hierarchy stays sparse. Once  $L_t$  falls below the floor ( $L_t < \tilde{L}$ ), subdividing the worst-case voxels can no longer reduce their complexity, so the extra leaves merely store redundant indices, and this redundancy grows as  $L_t \rightarrow 0$ , where every boundary voxel is refined to the maximum depth. *ManySpheres* makes the effect concrete, as its worst-case complexity stays at  $\tilde{L} = 1$  for  $L_t \leq 2$ , yet the index storage rises from 0.30 MB at  $L_t = 2$  to 107.04 MB at  $L_t = 0$ . Nevertheless, this redundancy inflates the memory but does not signal wasted evaluation. The average local complexity keeps falling as more voxels become uniform and the remaining boundary expressions grow simpler.



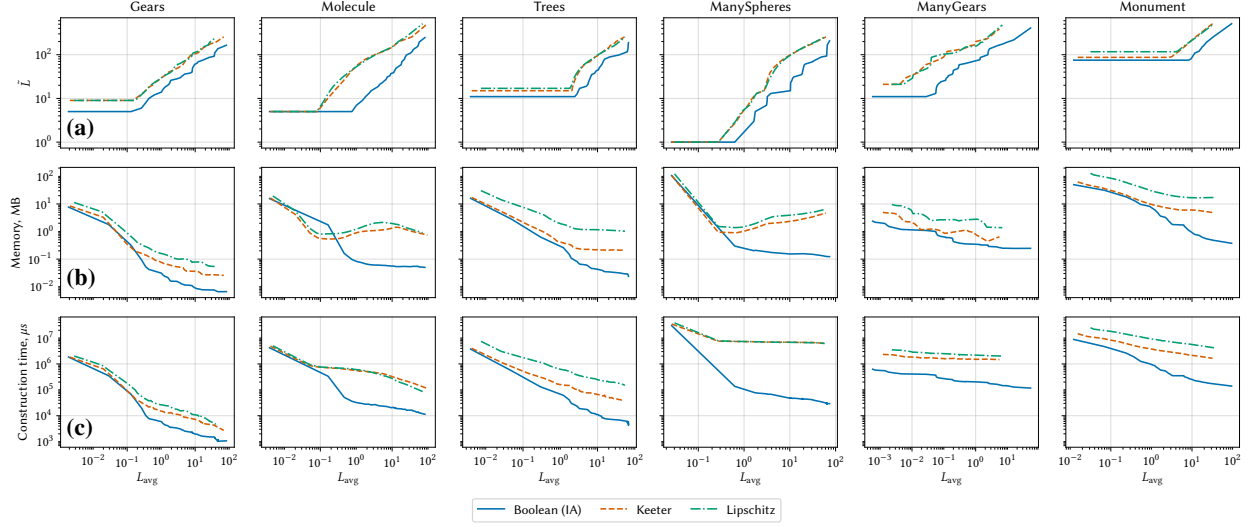
**Figure 8.** Per-pixel local complexity at leaf voxels (upper triangle) and number of traversed voxels (lower triangle) for the *Gears*, *ManySpheres*, and *Monument* scenes under different thresholds  $L_t$ . Average values over all non-background pixels are shown at the corners.

This steady improvement in local complexity does not carry over to runtime, because the average query time  $\bar{T}$  is not monotonic in  $L_t$  and the extra memory and construction time spent at low thresholds do not yield a matching runtime gain. At large thresholds ( $L_t \gtrsim 32$ ), traversal is negligible and  $\bar{T}$  is essentially the evaluation cost  $T_2$  (Eq. (2)), so it decreases nearly linearly with  $L_t$ . As  $L_t \rightarrow 0$ , the deeper hierarchy makes the traversal cost  $T_1$  significant, and the two effects balance at an optimum near  $L_t = 2$ –4. This optimum is shallow, as across small positive thresholds the query time stays close to its minimum, so most of the achievable speedup is realized well before  $L_t = 0$ . *ManyGears* behaves differently, showing little improvement across the tested range. As with the memory behavior described earlier, its large domain leaves  $K_{\max} = 10$  too shallow for aggressive pruning, so lowering  $L_t$  neither simplifies the worst-case voxels nor deepens the hierarchy appreciably. *ManySpheres* is the clearest demonstration of excessive subdivision at  $L_t = 0$ , where  $\bar{T}$  is almost twice as high as at  $L_t = 2$ . Overall, across all benchmarks, the conventional max-depth case never gives the best runtime and always incurs the largest index storage and leaf count, and the longest construction time.

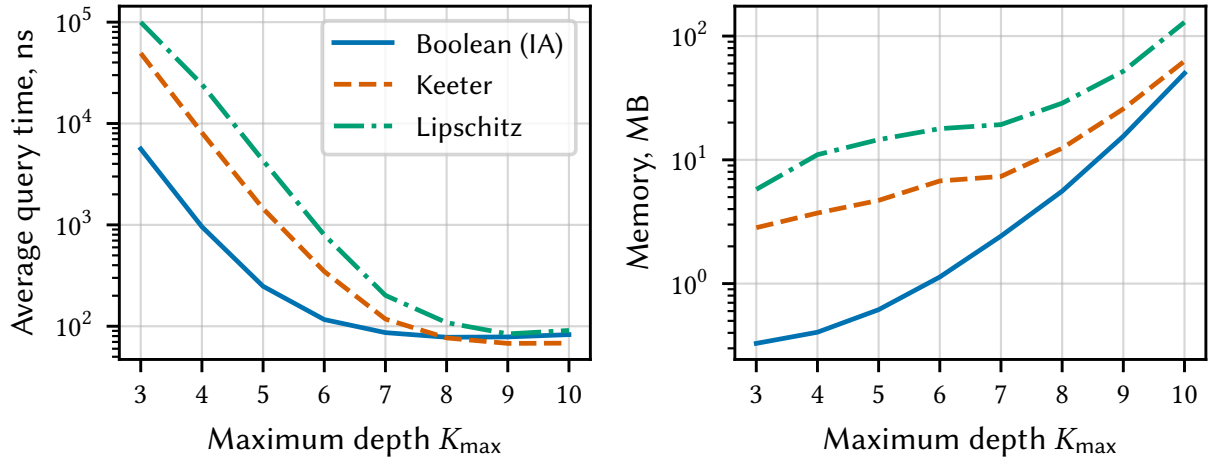
Rendering time exhibits a similar trend, since each pixel combines hierarchy traversal with repeated evaluations of the local expression during ray-casting and normal estimation. Large thresholds leave expensive expressions in coarse boundary voxels, while  $L_t = 0$  creates the most aggressive spatial refinement and therefore the largest traversal burden. The best rendering times shown in Table 2 are achieved at small positive thresholds, typically  $L_t = 2$ . Figure 8 illustrates the trade-off between the number of traversed voxels and the local complexity at leaf voxels for different thresholds. Moderate thresholds reduce the local complexity without forcing unnecessary refinement in regions where the local expression is already simple.

## 6.2 Comparison of pruning criteria

The three criteria are compared under the same threshold-based construction, sweeping the threshold over  $L_t \in [0, 500]$  at fixed maximum depth  $K_{\max} = 10$ . The Lipschitz far-field parameter is set to  $C = 1$ , which yields the best construction time and the smallest memory. The three metrics (maximum residual complexity,



**Figure 9.** Comparison of maximum residual complexity  $\tilde{L}$  (a), index storage (b), and construction time (c) as functions of average local complexity  $L_{\text{avg}}$ .



**Figure 10.** Effect of the maximum depth  $K_{\text{max}}$  on average query time and index storage for the Boolean, Keeter, and Lipschitz criteria on the *Monument* scene at threshold  $L_t = 0$ .

index storage, and construction time) are each measured at matched spatially averaged local complexity  $L_{\text{avg}}$  over the leaf voxels, a proxy for the per-query evaluation cost. Results for all scenes are shown in Figure 9.

**Maximum residual complexity.** Figure 9 (a) reports the maximum residual complexity  $\tilde{L}$  as a function of  $L_{\text{avg}}$ . Across the operating range, for a matched  $L_{\text{avg}}$  the Boolean criterion yields an  $\tilde{L}$  no larger than that of Keeter or Lipschitz, and typically smaller. The three curves converge only as  $L_t \rightarrow 0$ , where  $\tilde{L}$  saturates at the same unprunable floor  $L_{\text{min}}$  for all of them, since the worst voxel can no longer be simplified regardless of the criterion. This convergence holds only when  $K_{\text{max}}$  is deep enough to resolve the scene. Otherwise, the Boolean criterion is still more effective, as the large voxels in Figure 4 illustrate. The effect is clearest in *ManyGears*, where Keeter and Lipschitz yield an  $\tilde{L}$  roughly twice that of the Boolean criterion.

**Index storage and construction time.** Figure 9 (b) and (c) report the index storage and the octree construction time. These two metrics are strongly correlated, as both scale with the size of the hierarchy. A larger leaf count entails storing more voxel data and pruning more voxels during construction. At a matched  $L_{\text{avg}}$ , the Boolean criterion produces the sparsest hierarchy and correspondingly the lowest construction time, and it usually achieves the lowest index storage among the three criteria. The gap is widest at large  $L_{\text{avg}}$  (large  $L_t$ ) and narrows as  $L_t \rightarrow 0$ . In this limit early termination has little effect, since every criterion subdivides the boundary voxels down to the maximum depth and the resulting hierarchies coincide. At that resolution most voxels reduce to a single primitive, a floor no method can beat, so the more aggressive pruning has no further room to help. This convergence, however, relies on  $K_{\text{max}}$  being deep enough to resolve the scenes. Figure 10 isolates the role of  $K_{\text{max}}$  by sweeping it at fixed  $L_t = 0$  on *Monument*. For large  $K_{\text{max}}$  the three criteria are again nearly identical, but as  $K_{\text{max}}$  decreases the Boolean criterion pulls ahead. Its query time stays almost constant from  $K_{\text{max}} = 10$  down to 6, even though the index storage is roughly 50 $\times$  smaller. At this shallow depth the advantage is also largest. At  $K_{\text{max}} = 6$ , the Boolean criterion is roughly 3 $\times$  faster than Keeter and 7 $\times$  faster than Lipschitz.

## 7 Discussion

Table 2 shows that the best runtime is reached at intermediate thresholds, which raises the practical question of how the threshold should be chosen in advance. The proposed threshold-based stopping rule makes this a one-time, scene-independent choice. The maximum depth  $K_{\text{max}}$  is fixed once, sized to the most complex model in a collection, and a single intermediate threshold is then reused across all models without re-tuning. Subdivision stops as soon as the local complexity falls below the threshold, and  $K_{\text{max}}$  therefore serves only as an upper bound. Choosing a large  $K_{\text{max}}$  is safe, since simpler models terminate at coarser depths and do not incur the memory and traversal overhead that arises under the conventional max-depth termination rule.

The precise threshold that minimizes runtime does vary between scenes and cannot be determined analytically in advance. It depends on the scene, on  $K_{\text{max}}$ , and on implementation-specific costs. The arithmetic bound  $S_{\text{est}}$  (Eq. (6)) is not helpful because it ignores traversal cost and is conservative by construction, and therefore cannot identify the runtime optimum. This variability is immaterial in practice, because performance is flat over a broad intermediate range rather than peaked at a single value. Across the benchmark scenes, thresholds of roughly  $2 \leq L_t \leq 8$  yield near-identical query and rendering times (Table 2). The choice is also bounded from below. A non-empty voxel cannot be pruned below a single primitive, so  $L_t = 1$  is the most aggressive meaningful target. Reducing it to zero only forces additional subdivision. Across all six benchmark scenes,  $L_t = 0$  never yields the best runtime and always incurs the largest memory. A fixed threshold from the intermediate range therefore lands near the optimum on every scene, so the exact, scene-dependent optimum need not be located. Additionally, the threshold can be adjusted to meet the memory and performance requirements of the application.

The *ManySpheres* result deserves emphasis, as it exposes the regime in which the approach is most effective. The spheres are spatially disjoint, so any small voxel is governed by a single primitive, and the global complexity  $N = 5487$  collapses to a local complexity of one. The method should therefore deliver its largest gains on models in which many primitives are distributed across space with little local overlap, such as lattice microstructures, particle- and blob-based surfaces, and instanced assemblies. Such models have enormous global complexity, yet after pruning their evaluation cost and memory decouple almost entirely from  $N$ .

Beyond the specific pipeline studied here, the threshold-based stopping rule is straightforward to add to existing spatial-hierarchy frameworks for implicit objects. It requires only two ingredients: a measure of local complexity for each voxel, obtained from the pruned local expression, and a termination test that stops subdivision once this complexity falls below the threshold. Frameworks that already prune, such

as those built on interval or Lipschitz pruning, therefore have everything they need. With the hierarchy parameters fixed (for example, the maximum depth or subdivision factor), a single threshold then governs the construction, which makes the resulting memory and performance far more transparent and controllable than tuning those parameters directly. Much of the practical value lies in this simplicity, as the rule requires only minimal change and a single, interpretable parameter.

## 8 Future work

**Smooth CSG operators** The Boolean criterion is restricted to hard operators. Extending it to smooth CSG operators, which are already supported by the value-preserving criteria, is a natural next step. A promising direction is bounded blending Pasko et al. (2002), which confines the blend to a bounded region. Outside this region the operator reduces to a hard CSG operator, so the Boolean criterion applies as before. The main difficulty is structural, since a subexpression can be pruned only if no smooth operator lies on its path to the root. The complexity-driven threshold rule itself remains unchanged, and only the pruning criterion needs to adapt.

**GPU implementation** The present evaluation relies on a single-threaded CPU reference implementation. A GPU port is a natural next step that would broaden the impact of the method, since many modern applications of implicit surfaces are GPU-based. Per-voxel evaluation reduces to interpreting a compact token list, which makes it well-suited for the GPU tape-interpretation pipelines already demonstrated for interval pruning and Lipschitz pruning. Achieving this efficiently still requires careful treatment of GPU memory layout, branch divergence, and atomic updates during construction. A full GPU realization is therefore regarded as a promising direction for future work.

## 9 Conclusions and limitations

This work presents an acceleration pipeline for implicit CSG models that uses local complexity to guide subdivision, thereby shifting hierarchy construction from a geometry-driven to a complexity-driven paradigm. The proposed strategy therefore controls the evaluation cost of the pruned local expressions, enabling significant memory reductions while maintaining or improving query and rendering times compared to the conventional max-depth approach. The threshold-based construction is criterion-agnostic, as both sign-preserving and value-preserving pruning criteria are compatible with it. Among the criteria compared, the default Boolean criterion yields the sparsest hierarchies with the lowest worst-case complexity. As a result, the geometric resolution is decoupled from evaluation cost in detailed CSG models, which provides a flexible mechanism to control the memory–performance trade-off for the interrogation of high-fidelity implicit shapes.

The proposed approach has several limitations. At the framework level, the hierarchy is built in a preprocessing pass and the current single-threaded CPU reference implementation targets static scenes. A parallel GPU realization remains future work. The complexity-driven termination rule is limited by the intrinsic structure of the model. In scenes with dense overlapping features or geometric singularities, the maximum unprunable complexity  $L_{\min}$  remains high, so lowering the threshold forces excessive subdivision there without reducing complexity. Coarsening leaf voxels whose refinement would not lower the average complexity could mitigate this problem. Finally, richer operations beyond the CSG operators considered here, such as offsetting or warping, would require extending the pruning rules.

## References

- John Amanatides and Andrew Woo. A Fast Voxel Traversal Algorithm for Ray Tracing. In *EG 1987-Technical Papers*. Eurographics Association, 1987. doi: 10.2312/egtp.19871000.
- Wilhem Barbier, Mathieu Sanchez, Axel Paris, Élie Michel, Thibaud Lambert, Tamy Boubekeur, Mathias Paulin, and Theo Thonat. Lipschitz pruning: Hierarchical simplification of primitive-based sdfs. *Computer Graphics Forum*, 44(2):e70057, 2025. doi: <https://doi.org/10.1111/cgf.70057>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.70057>.
- Tom Duff. Interval arithmetic recursive subdivision for implicit functions and constructive solid geometry. *SIGGRAPH Comput. Graph.*, 26(2):131–138, July 1992. ISSN 0097-8930. doi: 10.1145/142920.134027. URL <https://doi.org/10.1145/142920.134027>.
- Eva Dyllong and Cornelius Grimm. A reliable extended octree representation of csg objects with an adaptive subdivision depth. In Roman Wyrzykowski, Jack Dongarra, Konrad Karczewski, and Jerzy Wasniewski, editors, *Parallel Processing and Applied Mathematics*, pages 1341–1350, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg. ISBN 978-3-540-68111-3.
- Mark T. Enszt, Duane W. Storti, and Mark A. Ganter. Implicit methods for geometry creation. *International Journal of Computational Geometry & Applications*, 08(05n06):509–536, 1998. doi: 10.1142/S0218195998000266. URL <https://doi.org/10.1142/S0218195998000266>.
- Sarah F. Frisken, Ronald N. Perry, Alyn P. Rockwood, and Thouis R. Jones. Adaptively sampled distance fields: a general representation of shape for computer graphics. In *Proceedings of the 27th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '00, page 249–254, USA, 2000. ACM Press/Addison-Wesley Publishing Co. ISBN 1581132085. doi: 10.1145/344779.344899. URL <https://doi.org/10.1145/344779.344899>.
- Oleg Fryazinov, Alexander Pasko, and Peter Comninou. Fast reliable interrogation of procedurally defined implicit surfaces using extended revised affine arithmetic. *Computers & Graphics*, 34(6):708–718, 2010. ISSN 0097-8493. doi: <https://doi.org/10.1016/j.cag.2010.07.003>. URL <https://www.sciencedirect.com/science/article/pii/S009784931000107X>.
- Eric Galin, Eric Guérin, Axel Paris, and Adrien Peytavie. Segment tracing using local lipschitz bounds. *Computer Graphics Forum*, 39(2):545–554, 2020. doi: <https://doi.org/10.1111/cgf.13951>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.13951>.
- John C. Hart. Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces. *The Visual Computer*, 12(10):527–545, 1996. doi: 10.1007/s003710050084.
- Rama Karl Hoetzlein. GVDB: Raytracing Sparse Voxel Database Structures on the GPU. In Ulf Assarsson and Warren Hunt, editors, *Eurographics/ ACM SIGGRAPH Symposium on High Performance Graphics*. The Eurographics Association, 2016. ISBN 978-3-03868-008-6. doi: 10.2312/hpg.20161197.
- Pierre Hubert-Brierre, Eric Guérin, Adrien Peytavie, and Eric Galin. Accelerating signed distance functions. *Computer Graphics Forum*, 44(7):e70258, 2025. doi: <https://doi.org/10.1111/cgf.70258>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.70258>.
- Tao Ju, Frank Losasso, Scott Schaefer, and Joe Warren. Dual contouring of hermite data. In *Proceedings of the 29th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '02, page 339–346, New York, NY, USA, 2002. Association for Computing Machinery. ISBN 1581135211. doi: 10.1145/566570.566586. URL <https://doi.org/10.1145/566570.566586>.



- Matthew J. Keeter. Massively parallel rendering of complex closed-form implicit surfaces. *ACM Trans. Graph.*, 39(4), August 2020. ISSN 0730-0301. doi: 10.1145/3386569.3392429. URL <https://doi.org/10.1145/3386569.3392429>.
- A. Knoll, Y. Hijazi, A. Kensler, M. Schott, C. Hansen, and H. Hagen. Fast ray tracing of arbitrary implicit surfaces with interval and affine arithmetic. *Computer Graphics Forum*, 28(1):26–40, 2009. doi: <https://doi.org/10.1111/j.1467-8659.2008.01189.x>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1467-8659.2008.01189.x>.
- Samuli Laine and Tero Karras. Efficient sparse voxel octrees. In *Proceedings of the 2010 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*, I3D '10, page 55–63, New York, NY, USA, 2010. Association for Computing Machinery. ISBN 9781605589398. doi: 10.1145/1730804.1730814. URL <https://doi.org/10.1145/1730804.1730814>.
- Charles E. Leiserson and Harald Prokop. Using de bruijn sequences to index a 1 in a computer word. Unpublished manuscript, June 1998.
- William E. Lorensen and Harvey E. Cline. Marching cubes: A high resolution 3d surface construction algorithm. *SIGGRAPH Comput. Graph.*, 21(4):163–169, August 1987. ISSN 0097-8930. doi: 10.1145/37402.37422. URL <https://doi.org/10.1145/37402.37422>.
- Kyoko Makino and Martin Berz. Taylor models and other validated functional inclusion methods. *International Journal of Pure and Applied Mathematics*, 4(4):379–456, 2003.
- Evgenii Maltsev, Dmitry Popov, Svyatoslav Chugunov, Alexander Pasko, and Iskander Akhatov. An accelerated slicing algorithm for frep models. *Applied Sciences*, 11(15), 2021. ISSN 2076-3417. doi: 10.3390/app11156767. URL <https://www.mdpi.com/2076-3417/11/15/6767>.
- R.E. Moore. *Interval Analysis*. Prentice-Hall series in automatic computation. Prentice-Hall, 1966. URL <https://books.google.fr/books?id=csQ-AAAAIAAJ>.
- Ken Museth. Vdb: High-resolution sparse volumes with dynamic topology. *ACM Trans. Graph.*, 32(3), July 2013. ISSN 0730-0301. doi: 10.1145/2487228.2487235. URL <https://doi.org/10.1145/2487228.2487235>.
- Ken Museth. Nanovdb: A gpu-friendly and portable vdb data structure for real-time rendering and simulation. In *ACM SIGGRAPH 2021 Talks*, SIGGRAPH '21, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383738. doi: 10.1145/3450623.3464653. URL <https://doi.org/10.1145/3450623.3464653>.
- Alexander Pasko, Valery Adzhiev, Alexei Sourin, and Vladimir Savchenko. Function representation in geometric modeling: concepts, implementation and applications. *The Visual Computer*, 11:429–446, 08 1995. doi: 10.1007/BF02464333.
- Alexander Pasko, Oleg Fryazinov, Turlif Vilbrandt, Pierre-Alain Fayolle, and Valery Adzhiev. Procedural function-based modelling of volumetric microstructures. *Graph. Models*, 73(5):165–181, September 2011. ISSN 1524-0703. doi: 10.1016/j.gmod.2011.03.001. URL <https://doi.org/10.1016/j.gmod.2011.03.001>.
- G. Pasko, A. Pasko, M. Ikeda, and T. Kunii. Bounded blending operations. In *Proceedings SMI. Shape Modeling International 2002*, pages 95–103, 2002. doi: 10.1109/SMI.2002.1003533.

- Dmitry Popov, Yulia Kuzminova, Evgenii Maltsev, Stanislav Evlashin, Alexander Safonov, Iskander Akhatov, and Alexander Pasko. Cad/cam system for additive manufacturing with a robust and efficient topology optimization algorithm based on the function representation. *Applied Sciences*, 11(16), 2021. ISSN 2076-3417. doi: 10.3390/app11167409. URL <https://www.mdpi.com/2076-3417/11/16/7409>.
- Aristides G. Requicha. Representations for rigid solids: Theory, methods, and systems. *ACM Comput. Surv.*, 12(4):437–464, December 1980. ISSN 0360-0300. doi: 10.1145/356827.356833. URL <https://doi.org/10.1145/356827.356833>.
- A. Ricci. A constructive geometry for computer graphics. *The Computer Journal*, 16(2):157–160, 01 1973. ISSN 0010-4620. doi: 10.1093/comjnl/16.2.157. URL <https://doi.org/10.1093/comjnl/16.2.157>.
- Marzia Riso, Élie Michel, Axel Paris, Valentin Deschaintre, Mathieu Gaillard, and Fabio Pellacini. Direct manipulation of procedural implicit surfaces. *ACM Trans. Graph.*, 43(6), November 2024. ISSN 0730-0301. doi: 10.1145/3687936. URL <https://doi.org/10.1145/3687936>.
- Vladimir L. Rvachev. Method of r-functions in boundary-value problems. *Soviet Applied Mechanics*, 11(4): 345–354, 1975. doi: 10.1007/BF00882900.
- John M. Snyder. Interval analysis for computer graphics. *SIGGRAPH Comput. Graph.*, 26(2):121–130, July 1992. ISSN 0097-8930. doi: 10.1145/142920.134024. URL <https://doi.org/10.1145/142920.134024>.
- J. Stolfi and L.H. de Figueiredo. An introduction to affine arithmetic. *Trends in Computational and Applied Mathematics*, 4(3):297–312, Jun. 2003. doi: 10.5540/tema.2003.04.03.0297. URL <https://tema.sbmec.org.br/tema/article/view/352>.
- Christopher Uchytel and Duane Storti. A function-based approach to interactive high-precision volumetric design and fabrication. *ACM Trans. Graph.*, 43(1), September 2023. ISSN 0730-0301. doi: 10.1145/3622934. URL <https://doi.org/10.1145/3622934>.
- Brian Wyvill, Andrew Guy, and Eric Galin. Extending the csg tree. warping, blending and boolean operations in an implicit surface modeling system. *Computer Graphics Forum*, 18(2):149–158, 1999. doi: <https://doi.org/10.1111/1467-8659.00365>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/1467-8659.00365>.
- Cédric Zanni. Synchronized tracing of primitive-based implicit volumes. *ACM Trans. Graph.*, 44(1), November 2024. ISSN 0730-0301. doi: 10.1145/3702227. URL <https://doi.org/10.1145/3702227>.

## A Surface gradient preservation

This appendix shows that the surface gradient is preserved under the Boolean criterion. For value-preserving criteria the result holds by construction, since the field is preserved. According to the Boolean rules in Table 1, the criterion discards any subtree whose removal does not affect the sign of the parent CSG expression within a voxel. Consequently, the zero level set of the CSG expression is preserved exactly, but the field values and gradients away from the surface may change. Nevertheless, for the hard CSG operators considered in this work, the surface gradient at regular surface points is preserved by the pruning, as shown below.

For min/max CSG, gradient preservation follows directly from local branch selection. With the F-rep convention (positive inside), union and intersection are

$$R_{\cup}(f, g) = \max(f, g), \quad R_{\cap}(f, g) = \min(f, g).$$

If  $g$  is removed by the union rule because  $g < 0$  throughout the voxel, then any regular surface point of the retained branch satisfies  $f = 0$  and  $g < 0$ . Therefore,

$$R_{\cup}(f, g) = \max(f, g) = f$$

locally at the surface, and

$$\nabla R_{\cup} = \nabla f.$$

Thus, after  $R_{\cup}(f, g)$  is replaced by  $f$ , the surface gradient is unchanged. The intersection case with  $g > 0$  is analogous:

$$R_{\cap}(f, g) = \min(f, g) = f, \quad \nabla R_{\cap} = \nabla f.$$

The cases where  $f$  is discarded follow by symmetry.

For the standard R-functions Rvachev (1975) used in F-rep, the corresponding operators are

$$R_{\cup}(f, g) = f + g + \sqrt{f^2 + g^2}, \quad R_{\cap}(f, g) = f + g - \sqrt{f^2 + g^2}.$$

Their gradients are

$$\begin{aligned} \nabla R_{\cup} &= \left(1 + \frac{f}{r}\right) \nabla f + \left(1 + \frac{g}{r}\right) \nabla g, \\ \nabla R_{\cap} &= \left(1 - \frac{f}{r}\right) \nabla f + \left(1 - \frac{g}{r}\right) \nabla g, \quad r = \sqrt{f^2 + g^2}. \end{aligned}$$

For the union case where  $g < 0$ , a regular surface point of the retained branch satisfies  $f = 0$ , so  $r = |g| = -g$ . Hence

$$\nabla R_{\cup} = \nabla f + \left(1 + \frac{g}{|g|}\right) \nabla g = \nabla f.$$

Thus, after replacing  $R_{\cup}(f, g)$  with  $f$ , the surface gradient is unchanged.

For the intersection case where  $g > 0$ , a regular surface point satisfies  $f = 0$ , so  $r = |g| = g$ . Hence

$$\nabla R_{\cap} = \nabla f + \left(1 - \frac{g}{|g|}\right) \nabla g = \nabla f.$$

Thus replacing  $R_{\cap}(f, g)$  with  $f$  also preserves the surface gradient.

The four pruning cases in which the retained branch can contain a regular surface point are summarized below.

| Operator   | Pruning condition | Surface gradient             |
|------------|-------------------|------------------------------|
| $f \cup g$ | $g < 0$           | $\nabla R_{\cup} = \nabla f$ |
| $f \cup g$ | $f < 0$           | $\nabla R_{\cup} = \nabla g$ |
| $f \cap g$ | $g > 0$           | $\nabla R_{\cap} = \nabla f$ |
| $f \cap g$ | $f > 0$           | $\nabla R_{\cap} = \nabla g$ |

Thus, any pruning condition that can affect a boundary voxel removes only a branch whose contribution to the surface gradient is zero. Applying the same argument recursively through the CSG tree shows that the pruned local expression has the same gradient as the original expression at regular surface points. Sharp features and other singular points, where the surface gradient is not uniquely defined, are excluded.

Preservation of higher-order derivatives depends on the operator encoding. For min/max operators, the retained branch coincides with the original field near any regular surface point, so all higher derivatives are preserved as well. For the standard R-functions, however, only the value and the gradient are preserved at the surface. For example, in the union case with  $f = 0$  and  $g < 0$ , the Hessian of the standard R-function is

$$\nabla^2 R_{\cup} = \nabla^2 f - \frac{\nabla f \nabla f^T}{g},$$

whereas the pruned expression yields simply  $\nabla^2 f$ . Therefore, Boolean pruning preserves the surface and its gradient at regular surface points, while the full Hessian is preserved only for the min/max encoding.